

BLM4522 – Ağ Tabanlı Paralel Dağıtım Sistemleri

Hasan Kalın - 20291267

GitHub: @hkln4

Drive Video: <https://drive.google.com/drive/folders/13OLp1ahdT-W1hLpW4e53MtYazaeDaf?usp=sharing>

PROJE 3: VERİTABANI GÜVENLİĞİ VE ERIŞİM KONTROLÜ

1. Giriş

Bu proje kapsamında PostgreSQL ve DBeaver kullanılarak bir İnsan Kaynakları veritabanı (hr_data – 1000 kayıt) üzerinde kapsamlı güvenlik önlemleri uygulandı. Rol tabanlı erişim yönetimi, pgcrypto ile veri şifreleme, SQL injection testleri, trigger tabanlı audit loglama ve pg_hba.conf ile güvenlik duvarı yönetimi gerçekleştirildi.

2.Erişim Yönetimi ve Roller

```
-- Roller oluştur
CREATE ROLE hr_readonly LOGIN PASSWORD 'readonly123';
CREATE ROLE hr_analyst LOGIN PASSWORD 'analyst123';
CREATE ROLE hr_manager LOGIN PASSWORD 'manager123';
CREATE ROLE hr_admin LOGIN PASSWORD 'admin123';

-- hr_readonly: sadece genel bilgilere erişim (hassas sütunlar hariç)
GRANT CONNECT ON DATABASE dbguvenligi_erisimkontrolu TO hr_readonly;
GRANT USAGE ON SCHEMA public TO hr_readonly;
GRANT SELECT (id, name, department, position, performance_score, joining_date)
ON hr_data TO hr_readonly;

-- hr_analyst: salary görebilir ama email/phone göremez
GRANT CONNECT ON DATABASE dbguvenligi_erisimkontrolu TO hr_analyst;
GRANT USAGE ON SCHEMA public TO hr_analyst;
GRANT SELECT (id, name, age, salary, department, position, performance_score, joining_date)
ON hr_data TO hr_analyst;

-- hr_manager: tüm sütunları görebilir, veri ekleyebilir/güncelleyebilir
GRANT CONNECT ON DATABASE dbguvenligi_erisimkontrolu TO hr_manager;
GRANT USAGE ON SCHEMA public TO hr_manager;
GRANT SELECT, INSERT, UPDATE ON hr_data TO hr_manager;

-- hr_admin: tam yetki
GRANT CONNECT ON DATABASE dbguvenligi_erisimkontrolu TO hr_admin;
GRANT USAGE ON SCHEMA public TO hr_admin;
GRANT ALL PRIVILEGES ON hr_data TO hr_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO hr_admin;

-- Roller doğrula
SELECT rolname, rolcanlogin, rolsuper
FROM pg_roles
WHERE rolname IN ('hr_readonly', 'hr_analyst', 'hr_manager', 'hr_admin');

-- Sütun bazlı yetkileri doğrula
SELECT grantee, column_name, privilege_type
FROM information_schema.column_privileges
WHERE table_name = 'hr_data'
AND grantee IN ('hr_readonly', 'hr_analyst', 'hr_manager', 'hr_admin')
ORDER BY grantee, column_name;
```

4 farklı kullanıcı rolü oluşturuldu. Her role sütun bazında farklı erişim yetkileri atandı. Hassas sütunlar (email, salary, phone_number) kısıtlı rollere kapatıldı.

Rol	Erişebildiği Sütunlar
hr_readonly	Id, name, department, position, score, joining_date
hr_analyst	+ age, salary (e-mail, phone yok)
hr_manager	Tüm sütunlar. INSERT, UPDATE
hr_admin	Tüm sütunlar. Tüm yetkiler

Rol	Giriş Yapabilir	Süper Kullanıcı
hr_readonly	True	False
hr_analyst	True	False
hr_manager	True	False
hr_admin	True	False

3. Veri Şifreleme

```
-- Script 2: Veri Şifreleme (pgcrypto)

-- pgcrypto extension'ını yükle
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Şifrelenmiş sütunlar için yeni tablo oluştur
ALTER TABLE hr_data
ADD COLUMN email_encrypted BYTEA,
ADD COLUMN phone_encrypted BYTEA,
ADD COLUMN salary_encrypted BYTEA;

-- Mevcut verileri şifrele (AES-256 ile)
UPDATE hr_data
SET
    email_encrypted = pgp_sym_encrypt(COALESCE(email, ''), 'gizli_anahtar_2024'),
    phone_encrypted = pgp_sym_encrypt(COALESCE(phone_number, ''), 'gizli_anahtar_2024'),
    salary_encrypted = pgp_sym_encrypt(COALESCE(salary::TEXT, ''), 'gizli_anahtar_2024');

-- Şifrelemeyi doğrula: şifreli ve çözülmüş halini karşılaştır
SELECT
    name,
    email AS original_email,
    pgp_sym_decrypt(email_encrypted, 'gizli_anahtar_2024') AS cozulmus_email,
    salary AS original_salary,
    pgp_sym_decrypt(salary_encrypted, 'gizli_anahtar_2024') AS cozulmus_salary
FROM hr_data
LIMIT 5;
```

pgcrypto extension'ı yüklenerek email, phone_number ve salary sütunları AES-256 algoritması ile şifrelendi. Şifreli veriler BYTEA tipinde ayrı sütunlarda saklandı. Şifre çözme işlemi yalnızca doğru anahtar ile mümkündür.

Sütun	Şifreleme Yöntemi	Şifreli Sütun Adı
email	pgp_sym_encrypt (AES-256)	email_encrypted
phone_number	pgp_sym_encrypt (AES-256)	phone_encrypted
salary	pgp_sym_encrypt (AES-256)	salary_encrypted

4. SQL Injection Testleri

```
-- Sadece grace'leri getirir, beklenen davranış
SELECT id, name, department, salary
FROM hr_data
WHERE TRIM(name) = 'grace';

-- Saldırgan 'grace' yerine bunu gönderdi
-- OR '1'='1' her zaman TRUE olduğu için WHERE koşulu devre dışı kalır
-- Tüm çalışanların salary bilgisi sızdı!
SELECT id, name, department, salary
FROM hr_data
WHERE TRIM(name) = 'grace' OR '1'='1';
```

Normal sorguda WHERE koşulu yalnızca 'grace' isimli 94 kaydı döndürdü.

Injection saldırısında OR '1'='1' eklenerek WHERE koşulu devre dışı bırakıldı ve tüm 1000 kayıt döndü ve hassas veriler sızdı.

```
-- KORUNMA YÖNTEMİ 1: Tek tırnak escape etme
-- Saldırganın girdisi artık SQL kodu değil, string olarak işlenir
SELECT id, name, department, salary
FROM hr_data
WHERE TRIM(name) = 'grace' OR '1'='1';
-- Sonuç: 0 satır - Injection etkisiz kaldı

-- KORUNMA YÖNTEMİ 2: Kullanıcı yetkilerini kısıtlama
-- hr_readonly sadece belirli sütunları görebilir
-- Injection başarılı olsa bile salary, email, phone göremez
SELECT grantee, column_name, privilege_type
FROM information_schema.column_privileges
WHERE table_name = 'hr_data'
AND grantee = 'hr_readonly'
ORDER BY column_name;

-- KORUNMA YÖNTEMİ 3: Girdi doğrulama
-- Sadece harf içeren name değerlerini kabul et
SELECT id, name, department, salary
FROM hr_data
WHERE TRIM(name) ~ '^([a-zA-Z ]+)$'
AND TRIM(name) = 'grace';
```

3 farklı korunma yöntemi uygulandı;

Saldırganın gönderdiği ' OR '1'='1' ifadesindeki tek tırnaklar escape edilerek (') SQL motoru tarafından SQL kodu olarak değil, düz metin olarak işlenmesi sağlandı. Bu yöntemde sorgu 0 satır döndürdü, injection tamamen etkisiz kaldı.

hr_readonly rolüne yalnızca belirli sütunlar için SELECT yetkisi verildi. Bu sayede injection saldırısı başarılı olsa bile saldırı salary, email ve phone_number

sütunlarına erişemez. Yetkisi olmayan bir sütunu sorgulamaya çalışıldığında PostgreSQL hata döndürür.

WHERE koşuluna TRIM(name) ~ '^([a-zA-Z]+)\$' regex filtresi eklendi. Bu filtre yalnızca harf ve boşluk içeren değerleri kabul eder. OR, =, ' gibi özel karakterler içeren injection girdileri bu koşulu sağlamadığı için sorgu yine yalnızca gerçek kayıtları döndürür.

5. Audit Logları

```
• 1. Audit için log tablosu oluşturun
CREATE TABLE audit_log (
  id SERIAL PRIMARY KEY,
  islem_zamani TIMESTAMP DEFAULT NOW(),
  kullanıcı TEXT DEFAULT CURRENT_USER,
  islem_turu TEXT,
  tablo_adi TEXT,
  etkilenen_id INTEGER,
  eski_deger TEXT,
  yeni_deger TEXT
);

• 2. hr_data tablosuna trigger fonksiyonu oluşturun
CREATE OR REPLACE FUNCTION audit_trigger_func()
RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO audit_log (islem_turu, tablo_adi, etkilenen_id, yeni_deger)
    VALUES ('INSERT', TG_TABLE_NAME, NEW.id,
      'name: ' || NEW.name || ', department: ' || NEW.department);
  ELSEIF TG_OP = 'UPDATE' THEN
    INSERT INTO audit_log (islem_turu, tablo_adi, etkilenen_id, eski_deger, yeni_deger)
    VALUES ('UPDATE', TG_TABLE_NAME, NEW.id,
      'salary: ' || OLD.salary,
      'salary: ' || NEW.salary);
  ELSEIF TG_OP = 'DELETE' THEN
    INSERT INTO audit_log (islem_turu, tablo_adi, etkilenen_id, eski_deger)
    VALUES ('DELETE', TG_TABLE_NAME, OLD.id,
      'name: ' || OLD.name || ', department: ' || OLD.department);
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

• 3. Trigger'ı hr_data tablosuna bağla
CREATE TRIGGER hr_data_audit
AFTER INSERT OR UPDATE OR DELETE ON hr_data
FOR EACH ROW EXECUTE FUNCTION audit_trigger_func();

• 4. Trigger'ı doğrula
SELECT trigger_name, event_manipulation, event_object_table
FROM information_schema.triggers
WHERE event_object_table = 'hr_data';

SELECT setval('hr_data_id_seq', (SELECT MAX(id) FROM hr_data));

• 5. Trigger'ı test et

-- INSERT testi
INSERT INTO hr_data (name, age, salary, gender, department, position, joining_date, performance_score, email, phone_number)
VALUES ('Test User', 30, 55000, 'Male', 'IT', 'Engineer', '2023-01-01', 'A', 'test@test.com', '1234567890');

-- UPDATE testi
UPDATE hr_data SET salary = 60000 WHERE name = 'Test User';

-- DELETE testi
DELETE FROM hr_data WHERE name = 'Test User';

• Audit log'u görüntüle
SELECT * FROM audit_log ORDER BY islem_zamani DESC LIMIT 10;
```

hr_data tablosuna bağlı bir trigger fonksiyonu oluşturuldu. INSERT, UPDATE ve DELETE işlemlerinin tamamı audit_log tablosuna otomatik olarak kaydedildi.

Alan	Açıklama
islem_zamani	İşlemin gerçekleştiği zaman damgası
kullanici	İşlemi yapan PostgreSQL kullanıcısı
islem_turu	INSERT / UPDATE / DELETE
tablo_adi	İşlem yapılan tablo
etkilenen_id	Etkilenen kaydın ID'si
eski_deger	Güncellemeden önceki değer
yeni_deger	Güncellemeden sonraki değer

Trigger test edildi, INSERT, UPDATE ve DELETE işlemleri sırasıyla loglandı:

- INSERT: name: Test User, department: IT
- UPDATE salary: 55000.00 → 60000.00
- DELETE name: Test User, department: IT

4	2026-05-03 01:47:52.749	hkl4_	INSERT	hr_data	1002	name: Test User, department: IT
5	2026-05-03 01:47:52.749	hkl4_	UPDATE	hr_data	1002	salary: 55000.00 salary: 60000.00
6	2026-05-03 01:47:52.749	hkl4_	DELETE	hr_data	1002	name: Test User, department: IT
1	2026-05-02 03:52:15.255	hkl4_	INSERT	hr_data	1001	name: Test User, department: IT
2	2026-05-02 03:52:15.255	hkl4_	UPDATE	hr_data	1001	salary: 55000.00 salary: 60000.00
3	2026-05-02 03:52:15.255	hkl4_	DELETE	hr_data	1001	name: Test User, department: IT

6. Güvenlik Duvarı Yönetimi

PostgreSQL'in istemci kimlik doğrulama dosyası pg_hba.conf düzenlenerek hr_* rolleri için scram-sha-256 şifreli kimlik doğrulama zorunlu hale getirildi. Öncesinde tüm bağlantılar şifresiz (trust) kabul ediliyordu.

Önceki Hali					Yeni Hali			
local	all	all	trust		local	all	hkl4_	trust
host	all	all	127.0.0.1/32	trust				
host	all	all	:::1/128	trust				
					local	dbguvenligi_erisimkontrolu	hr_readonly	scram-sha-256
					local	dbguvenligi_erisimkontrolu	hr_analyst	scram-sha-256
					local	dbguvenligi_erisimkontrolu	hr_manager	scram-sha-256
					local	dbguvenligi_erisimkontrolu	hr_admin	scram-sha-256
					local	all	all	trust
					host	all	all	127.0.0.1/32
					host	all	all	:::1/128

Değişiklik sonrası hr_readonly rolü ile bağlantı kurulmak istendiğinde şifre sorulduğu ve doğru şifre girilince bağlantının başarılı olduğu doğrulandı.

7. Sonuç

Bu proje kapsamında HR veritabanı üzerinde kapsamlı bir güvenlik altyapısı kuruldu. Rol tabanlı erişim kontrolü ile hassas sütunlar (email, salary, phone) yetkisiz kullanıcılara kapatıldı. pgcrypto ile bu sütunlar AES-256 ile şifrelenerek fiziksel veri güvenliği sağlandı. SQL injection saldırısının nasıl gerçekleştiği ve 3 farklı korunma yöntemi gösterildi. Trigger tabanlı audit log mekanizması ile tüm INSERT, UPDATE ve DELETE işlemleri otomatik olarak loglandı. Son olarak pg_hba.conf düzenlenerek şifresiz bağlantılar kaldırıldı ve scram-sha-256 kimlik doğrulama zorunlu hale getirildi.